OSSP (Operating Systems and Systems Programming) – 25CS2104E

OSSP LAB – SKILL-2

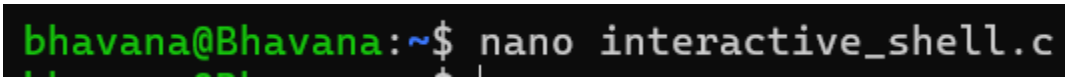
Aim

To develop and test an interactive command-line loop that displays a prompt, reads user input, handles exit conditions, captures keyboard input including backspace and Enter keys, manages the input buffer, and supports multi-character commands.

Algorithm

1. Start the program.
2. Display the command prompt.
3. Read characters entered by the user.
4. Store the characters in an input buffer.
5. If a normal character is entered, add it to the buffer.
6. If **Backspace** is pressed, remove the last character from the buffer.
7. If **Enter** is pressed, terminate the current input and process the command.
8. Check whether the entered command is an exit command.
9. If it is an exit command, terminate the loop and end the program.
10. Otherwise, process and display the entered command.
11. Return to the main loop and display the prompt again.
12. Test the loop with single-character and multi-character commands.
13. Stop the program.

Step 1



Step 2

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        // Display prompt
        printf("myshell> ");
        fflush(stdout);

        // Read user input
        fgets(input, sizeof(input), stdin);

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle exit condition
        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        // Display entered command
        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Step 3

```
bhavana@Bhavana:~$ ./interactive_shell
myshell> hello
You entered: hello
myshell> klh
You entered: klh
myshell>
```

Step 4

```
bhavana@Bhavana:~$ nano keyboard.c
```

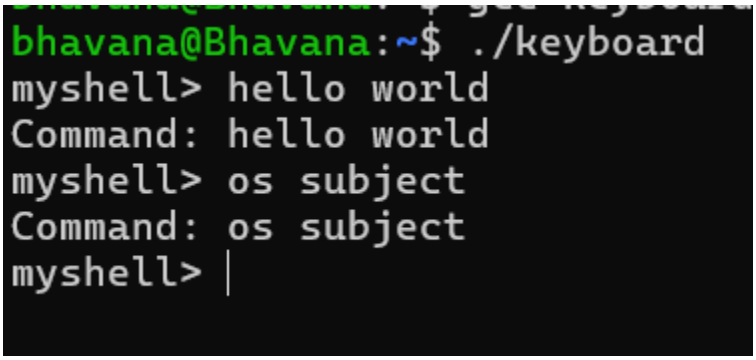
```c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

int main()
{
    struct termios old, new;
    char buffer[100];
    char ch;
    int i = 0;

    tcgetattr(STDIN_FILENO, &old);
    new = old;
    new.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &new);

    printf("myshell> ");

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n')
        {
            buffer[i] = '\0';

            if (strcmp(buffer, "exit") == 0)
                break;

            printf("\nCommand: %s\nmyshell> ", buffer);
            i = 0;
        }
        else if (ch == 127)
        {
            if (i > 0)
            {
```

```
^G Help          ^O Write Out     ^F Where Is      ^K Cut
^X Exit          ^R Read File     ^\ Replace       ^U Pas
```

Step 5

```
bhavana@Bhavana:~$ gcc keyboard.c -o keyboard
```

Step 6



Conclusion

The interactive command-line loop was successfully designed and tested. It correctly displays the prompt, accepts keyboard input, handles **Backspace and Enter**, manages the input buffer, supports multi-character commands, and terminates when an exit condition is received.